UNIVERSITÀ DEGLI STUDI DI BERGAMO
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione



Introduzione alla progettazione di algoritmi

PROGETTAZIONE, ALGORITMI E COMPUTABILITÀ (38090-MOD1)

Corso di laurea Magistrale in Ingegneria Informatica

RELATORE
Prof.ssa Patrizia Scandurra

SEDE
DIGIP

Algoritmi e problemi computazionali

Gli algoritmi forniscono la **strategia risolutiva** per giungere alla soluzione di un dato problema computazionale (o di calcolo)

Classificazione dei problemi:

- problemi non decidibili
  - es. l'*Halting program [Turing, 1937]*
    // restituisce true se l'algoritmo A su input finito d termina, false altrimenti
    `boolean halts(A, d)`

- problemi decidibili
  - problemi trattabili (costo polinomiale)
  - problemi intrattabili (costo esponenziale)

Algoritmi come soluzioni di problemi computazionali

- Esempio 1 (**Ordinamento**): Dati i risultati di una prova di esame, disporre i nomi degli studenti in ordine decrescente di voto
- Esempio 2 (**Calendario**): Calcolare il giorno della settimana corrispondente ad una certa data specificata da giorno, mese e anno.
- Esempio 3 (**Duplicati**): Data una lista di prenotazioni agli esami, verificare se vi sono studenti che si sono prenotati più di una volta
- Esempio 4 (**Ricerca**): ricercare i dati di un dato studente da una base di dati e/o collezione di record studenti, data la matricola come chiave di ricerca

Definizione di algoritmo

Una sequenza **finita** di operazioni **non ambigue** ed **effettivamente calcolabili** che, una volta eseguite, **producono un risultato** in una **quantità finita di tempo**

- Esempio: algoritmo `preparaCaffè`



algoritmo `preparaCaffè`

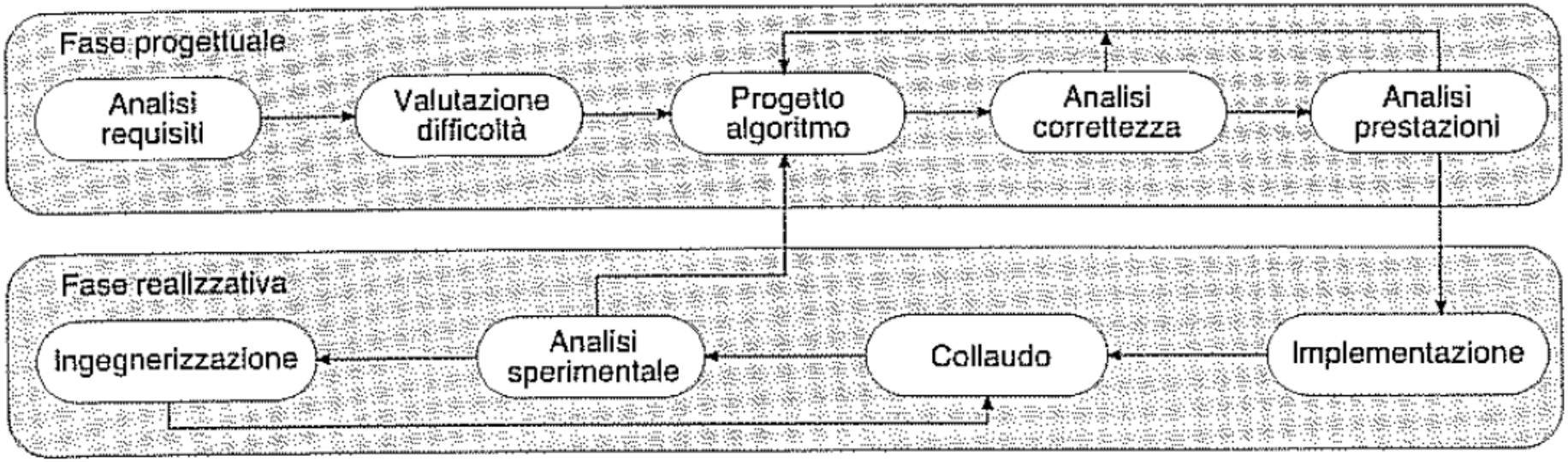
1. Svita la caffettiera.
2. Riempi d'acqua il serbatoio della caffettiera.
3. Inserisci il filtro.
4. Riempi il filtro con la polvere di caffè.
5. Avvita la parte superiore della caffettiera.
6. Metti la caffettiera, così predisposta, su un fornello acceso.
7. Spegni il fornello quando il caffè è pronto.
8. Versa il caffè nella tazzina.

Proprietà fondamentali di un algoritmo

- **Finitezza**: la sequenza di istruzioni deve essere finita (finitezza)
- **Non ambiguità**: le istruzioni devono essere espresse in modo non ambiguo
- **Realizzabilità**: le istruzioni devono essere eseguibili materialmente
- **Efficacia**: la sequenza di istruzioni deve portare ad un risultato
- **Efficienza**: È inoltre importante valutare le **risorse utilizzate** (tempo, memoria, …) perché un consumo eccessivo delle stesse può pregiudicare la possibilità stessa di utilizzo di un algoritmo

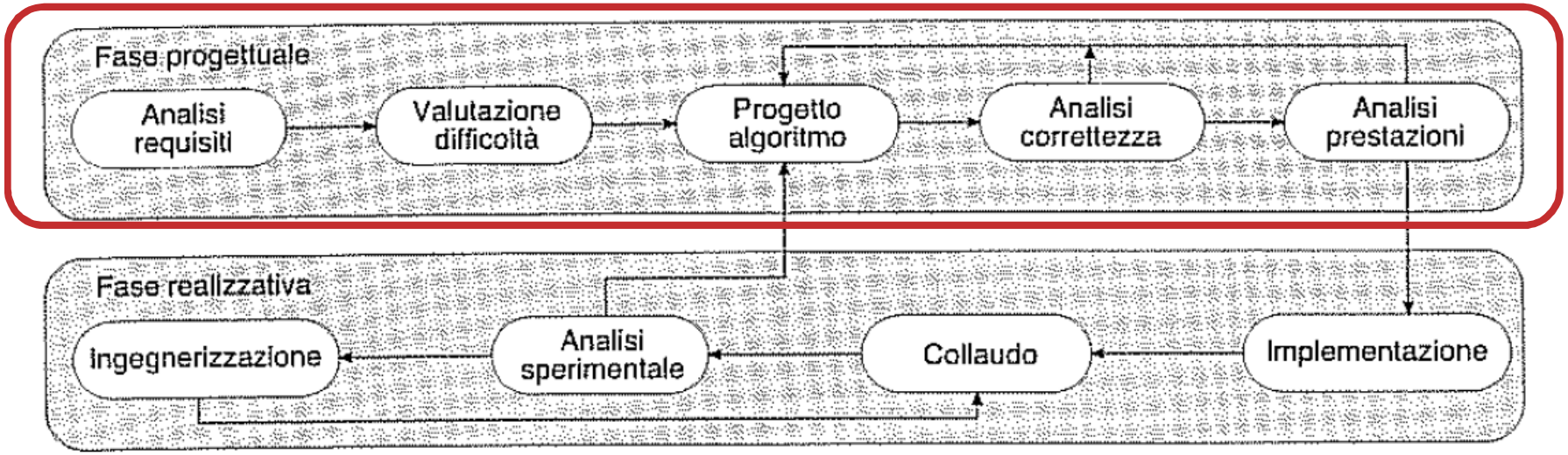
Ciclo di sviluppo di codice algoritmico

Modello ciclico a due fasi: **fase progettuale** e **fase realizzativa**

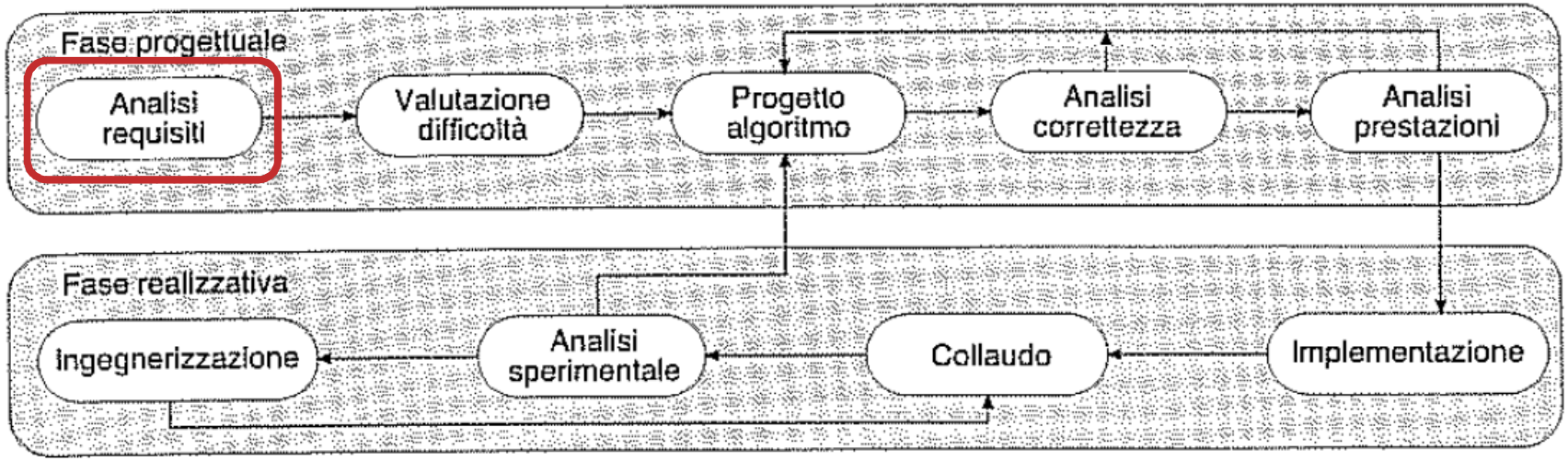


- **Fase progettuale**

design in piccolo!



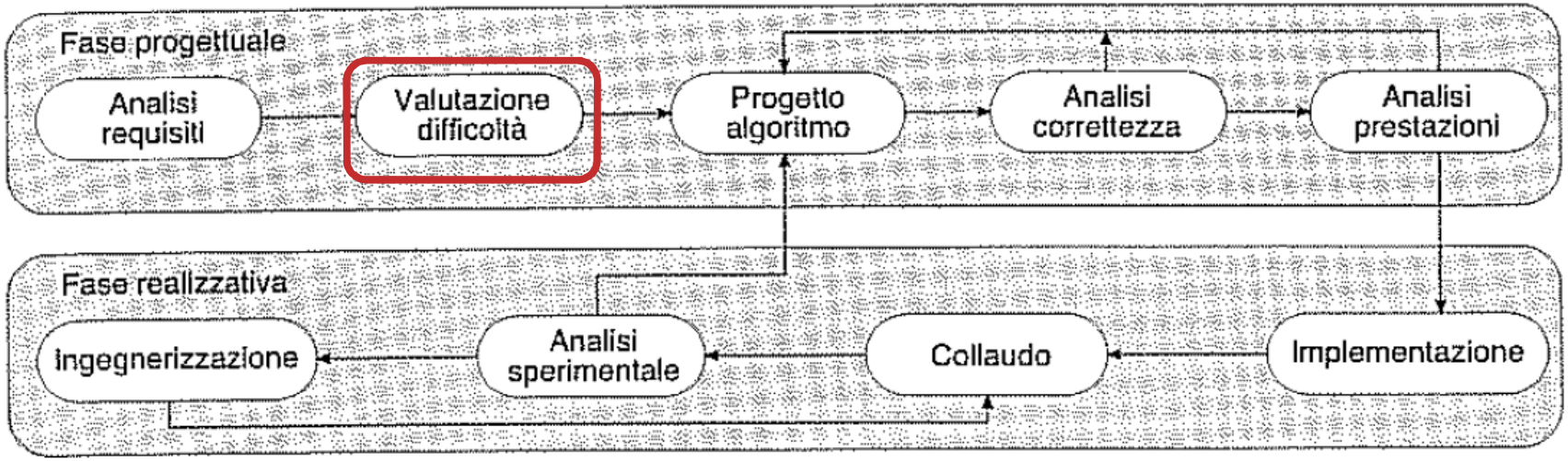
- **Fase progettuale**



Analisi dei requisiti

- **Definire il problema di calcolo** che si intende risolvere identificando:
  - i requisiti dei dati in ingresso
  - i requisiti dei dati in uscita prodotti dall'algoritmo
- Identificare una **metodologia di progettazione**:
  - Esempio: un problema complesso può essere scomposto in un certo numero di sotto-problemi risolvibili separatamente
    - L'approccio *divide-et-impera*

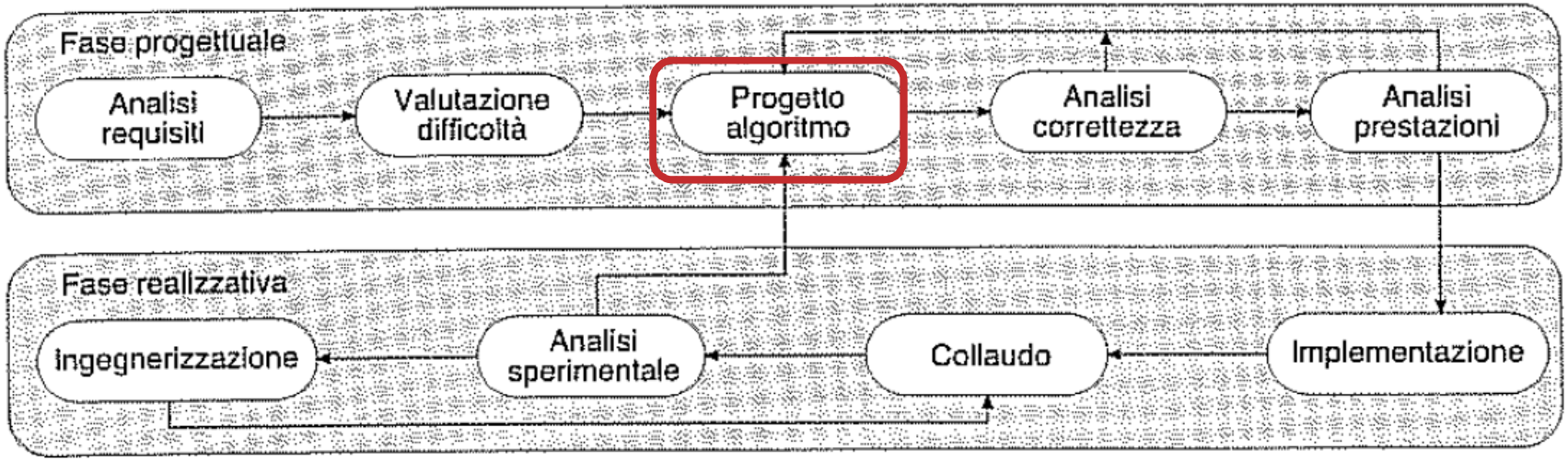
- **Fase progettuale**



Valutazione delle difficoltà

- Idealmente vorremmo algoritmi che minimizzino le risorse spazio/tempo utilizzate
  - **Complessità** (spaziale o temporale) **dell'algoritmo**
- Tuttavia, **per ogni problema computazionale esistono dei limiti inferiori** alle quantità di risorse di calcolo necessarie alla sua risoluzione
  - **Complessità del problema**
- Stimare tali limiti ci permette di capire se un dato algoritmo è genuino o se è sperabile poter fare meglio
  - *Esempi: ricerca di un elemento, ordinamento su confronto, duplicati*

- **Fase progettuale**



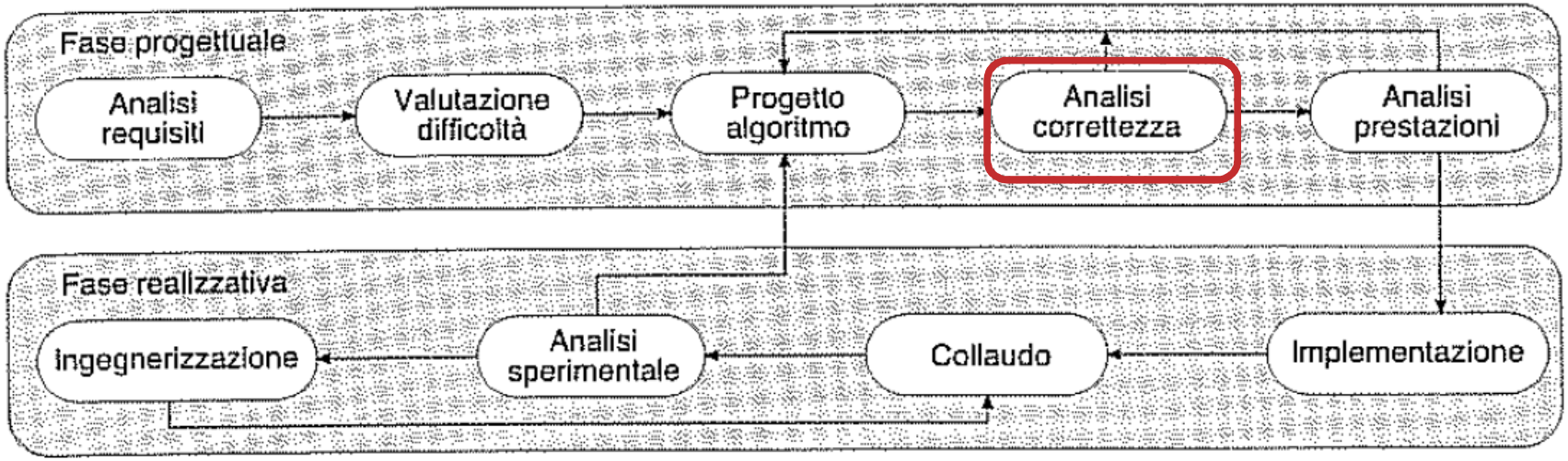
Progetto di un algoritmo risolutivo

- Richiede creatività ed esperienza
- Per mantenere il massimo grado di generalità, descriveremo gli algoritmi in pseudocodice:
  - ricorda linguaggi di programmazione reali come C, C++ o Java
  - può contenere alcune frasi in italiano



```
algoritmo verificaDup(sequenza S)
    for each elemento x della sequenza S do
        for each elemento y che segue x nella sequenza S do
            if x = y then return true
    return false
```

- **Fase progettuale**



Correttezza

Vogliamo progettare algoritmi che:

- Producano correttamente il risultato desiderato

Analisi di correttezza

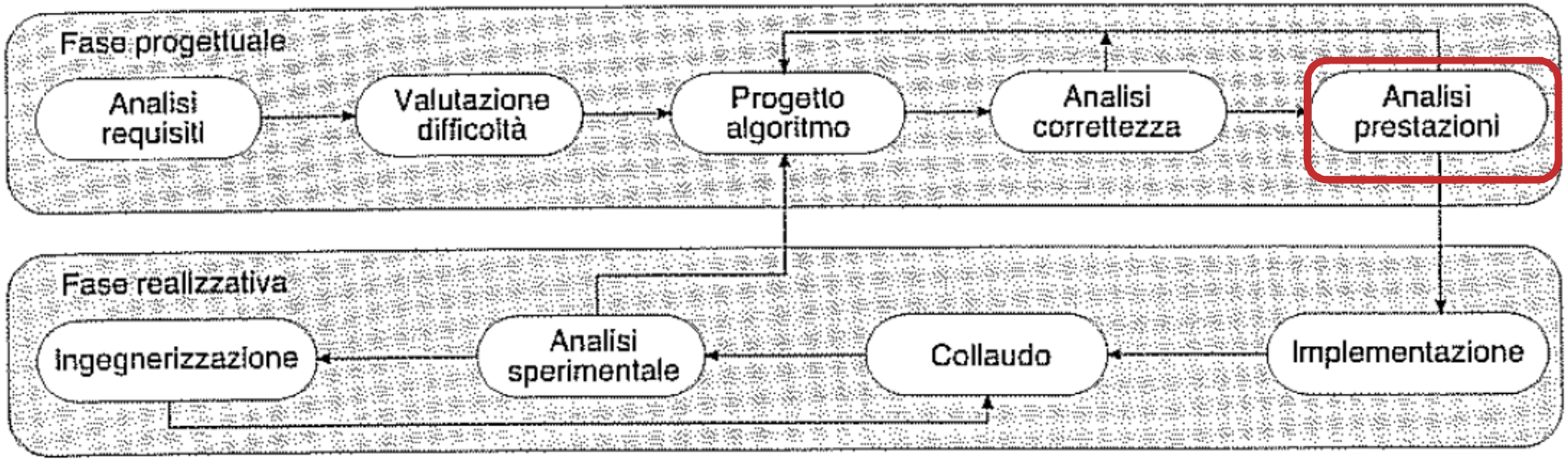
- **Verifica formale della correttezza**
  - prova che i requisiti sui dati prodotti dall'algoritmo (requisiti sull'output) siano soddisfatti
  - ad esempio, dimostrando che certe proprietà sui dati tenuti in memoria dall'algoritmo vengono mantenute ad ogni passo
- Ove l'analisi evidenzi discrepanze, è necessario ritornare alla fase di progettazione
- Esempio (**Duplicati**): Osserviamo che l'algoritmo confronta almeno una volta ogni coppia di elementi; quindi, se esiste un elemento che si ripete, verrà sicuramente trovato (true). Per lo stesso motivo, se l'algoritmo restituisce false, abbiamo invece la garanzia che tutte le coppie sono distinte.

Correttezza ed efficienza

Vogliamo progettare algoritmi che:

- Producano correttamente il risultato desiderato
- Siano efficienti in termini di:
  - tempo di esecuzione e occupazione di memoria

- **Fase progettuale**



Perché analizzare gli algoritmi

- **L'analisi teorica sembra essere più affidabile di quella sperimentale**: vale su tutte le possibili istanze di dati su cui l'algoritmo opera
- Ci aiuta a scegliere tra diverse soluzioni allo stesso problema
- Permette di **predire le prestazioni** di un programma software, prima ancora di scriverne le prime linee di codice

Analisi delle prestazioni

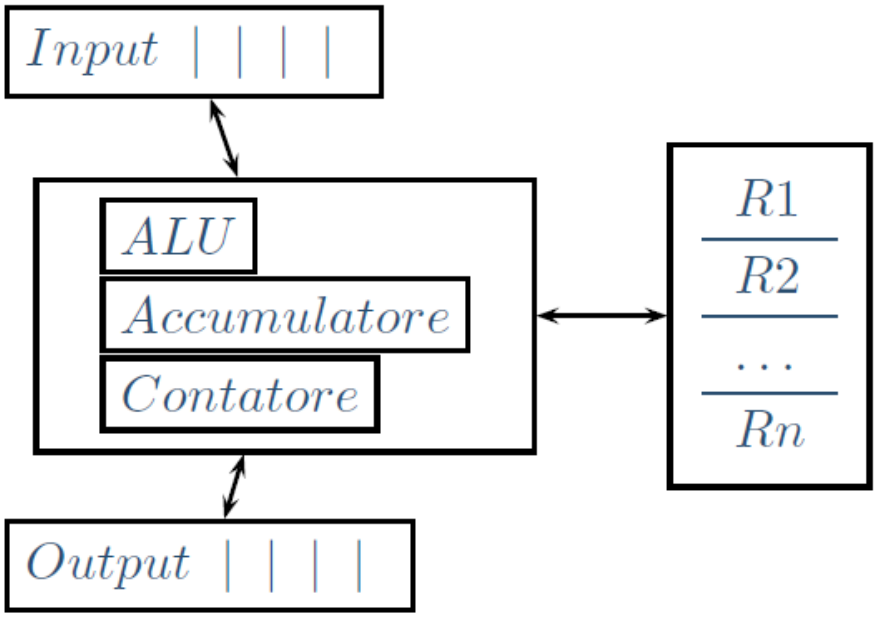
- Assumiamo di utilizzare *Mono-Processore + RAM (Random Access Memory):* assenza di concorrenza e parallelismo
- Analisi teorica condotta fissando:
  1. **Modello di calcolo** e
     - tipicamente la **macchina a registri RAM (Random Access Machine)**
       - si ispira all'architettura di Von Neumann e alla macchina di Turing
       - l'accesso diretto o casuale alla memoria, e non sequenziale come nella macchina di Turing
  2. **Modello di costo teorico**
     - **modello a costo uniforme** (indipendente dalla dimensione degli operandi coinvolti) – come prima approssimazione
     - **modello a costo logaritmico** (dipende dalla dimensione degli operandi coinvolti) – un po' complicato

Macchine a registri

Sono caratterizzate da:

- **Nastri** di input e di output
- Un numero arbitrario di celle *i* di **memoria** (detti ***registri***) contenenti un intero arbitrario: R[i] (intero o reale di dimensione arbitraria)
- ALU e registri speciali:
  - **Contatore**: l'indirizzo istruzione successiva
  - **Accumulatore R0**: l'operando su cui agisce l'istruzione corrente
- **Passo di calcolo**:
  - Istruzione di I/O su nastro
  - Istruzione di I/O da/in memoria
  - Istruzione aritmetica

  o logica



Il modello assume che l'algoritmo sia sequenziale e deterministico!

Modello a costo uniforme

- **Ogni istruzione elementare** della macchina a registri costa una **unità di tempo**
- **Ciascuna locazione di memoria**, compreso l'accumulatore, richiede **una unità di spazio**
- **Pregio**: approccio molto semplice
- **Difetto**: analisi non realistica
  - Moltiplicare due interi di 32 bit costa quanto moltiplicare due interi di 1000 bit
  - Accedere ad una memoria di 1 kilobyte costa come accedere ad una memoria di 1 terabyte

Modello a costo logaritmico

- La complessità del **calcolo** è proporzionale al numero di bit dell'operando e l'**accesso** è proporzionale al numero di bit dell'indirizzo
  - l'elaborazione di un intero n ha costo O(log n)
  - l'accesso ad un registro i ha costo O(log i)
- Si applica la medesima filosofia per l'occupazione di spazio
- **Pregio**: la valutazione dei costi è più realistica dal punto di vista asintotico (ovvero per operandi molto grandi, o per memoria molto grande)
- **Difetto**: analisi più laboriosa

Il nostro modello di costo

- **Limitiamo a c*log n bit la dimensione dei numeri interi rappresentabili**
  - c è una costante
  - n è la dimensione dell'input del problema
- **Non ammettiamo operazioni sui numeri in virgola mobile**
- Quindi gli algoritmi che presentiamo d'ora in poi sono implementabili su **una macchina a registri con interi di dimensione <= n^c**

Tempo di esecuzione

- Calcoliamo il numero di linee di codice mandate in esecuzione
  - misura indipendente dalla piattaforma utilizzata
  - ma approssimativa!
- Meglio considerare il numero di passi elementari eseguiti dall'algoritmo al variare della dimensione n dell'input: funzione T(n)
- Ci interessa soprattutto il comportamento di T(n) per valori "*grandi*" di *n* (**comportamento asintotico**)

Occupazione di memoria

- Se abbiamo un algoritmo lento, dovremo solo attendere più a lungo per ottenere il risultato

- Ma se un algoritmo richiede più spazio di quello a disposizione, non otterremo mai la soluzione, indipendentemente da quanto attendiamo

Calcolabilità e complessità

- **Calcolabilità**: nozioni di algoritmo e di problema non decidibile
- **Complessità**: nozione di algoritmo efficiente e di problema intrattabile

La calcolabilità ha lo scopo di classificare i problemi in risolvibili e non risolvibili, mentre la complessità in facili e difficili

Calcolo della complessità

- **Complessità di un algoritmo:** misura del numero di passi elementari che si devono eseguire per risolvere il problema
- **Complessità di un problema:** complessità del **migliore** algoritmo che lo risolve

in funzione della taglia (dimensione dell'input) del problema

Taglia di un problema

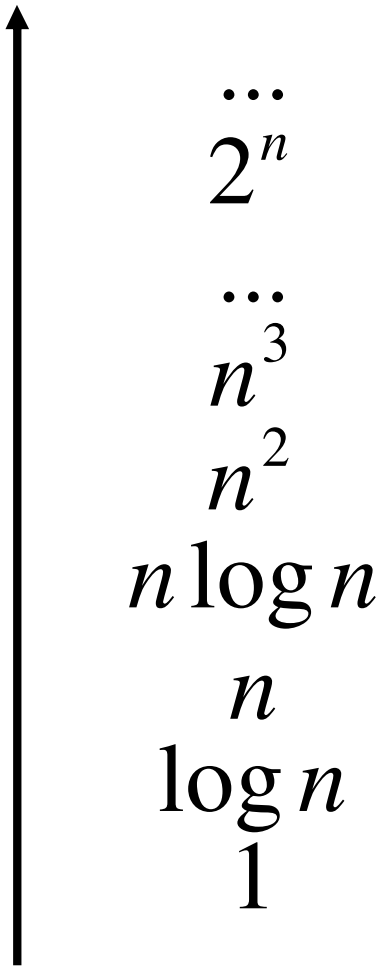
misura della dimensione dell'input (in bit, parole, ecc.)

- ordinamento: numero di oggetti da ordinare
- gestione dati: numero dei dati da gestire
- algoritmi e problemi su grafi: numero di archi e nodi del grafo

dipende dalla rappresentazione dei dati (struttura dati)

Paragonare tra loro algoritmi

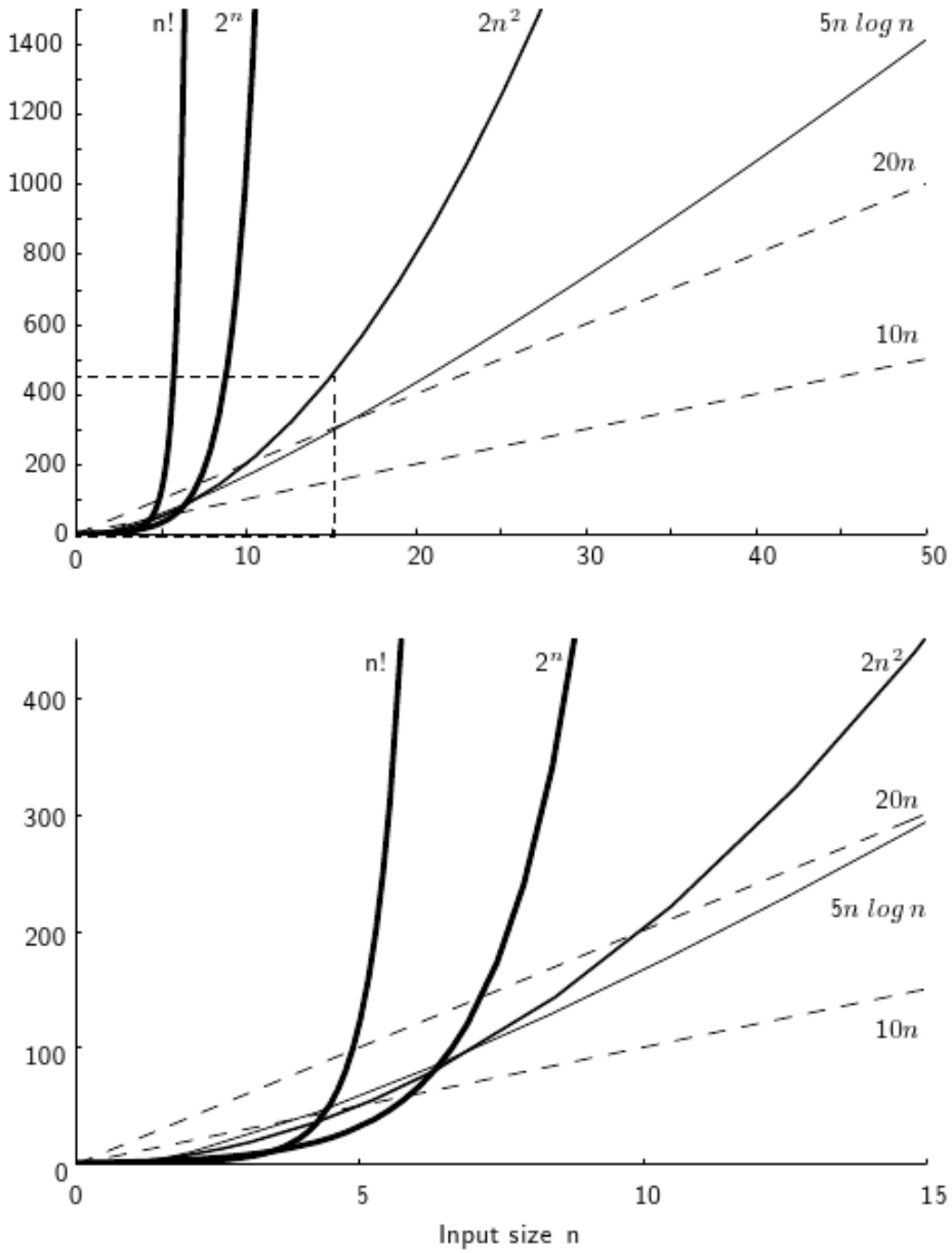
Abbiamo una scala di complessità:



vogliamo inserire ogni algoritmo in questa scala



Growth Rate


n!
2^n
$2n^2$
$5n\ log\ n$
$20n$
$10n$
1400
1200
1000
800
600
400
200
0
0
10
20
30
40
50
n!
2^n
$2n^2$
$20n$
$5n\ log\ n$
$10n$
400
300
200
100
0
0
5
10
15
Input size n

Algoritmi esponenziali

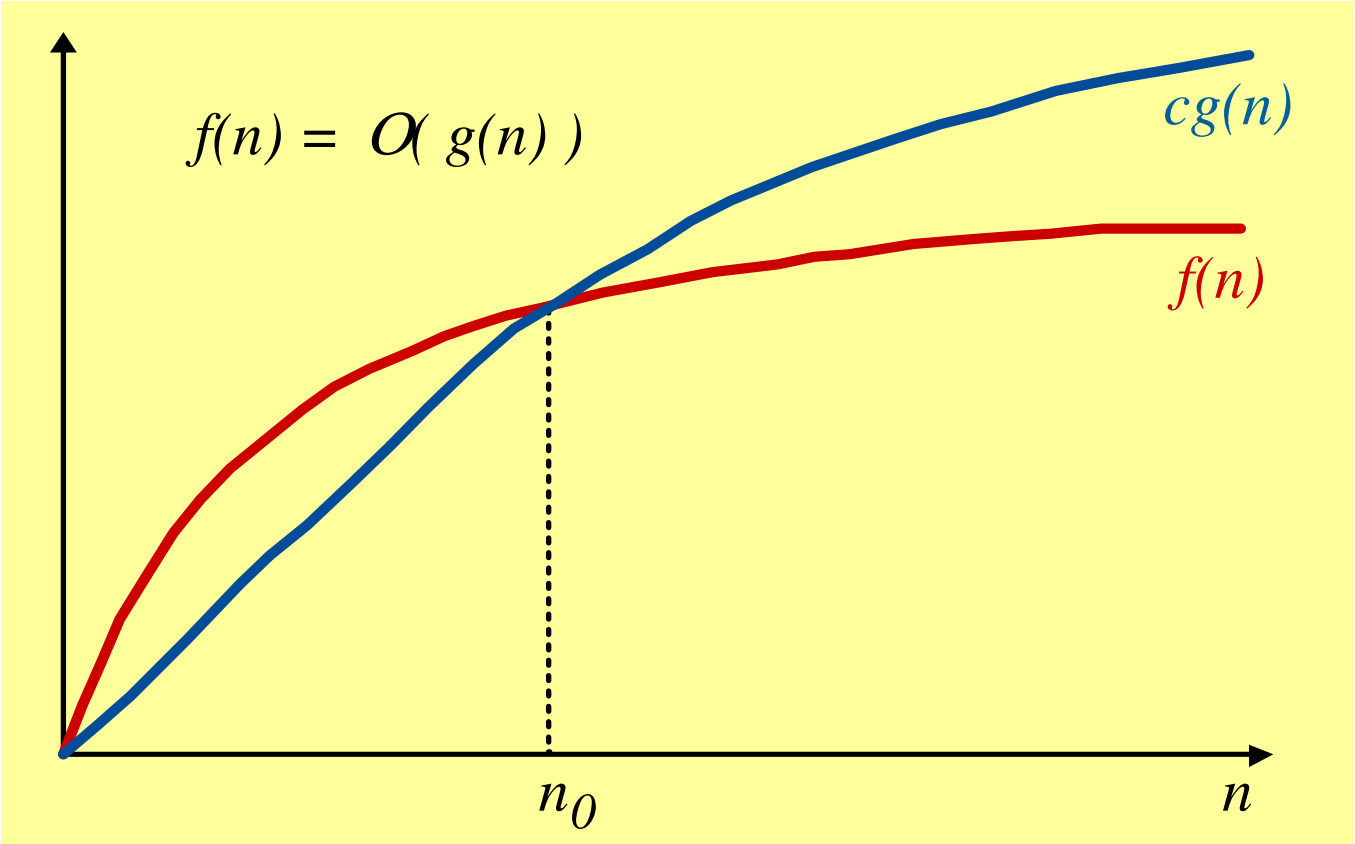
n	$\log_2(n)$	$100*n$	$10*n^2$	n^3	2^n
10	2.3 µs	1ms	1ms	1ms	1024µs ~ 1ms
20	2.99µs	2ms	4ms	8ms	1.048sec
60	4.09µs	6ms	36ms	0.21sec	2^{60}µs ~ 366 secoli

Notazione asintotica

- Per lo stesso programma impaginato diversamente potremmo concludere ad esempio che T(n)=3n oppure T(n)=5n
- Vorremmo **un modo per descrivere l'ordine di grandezza di T(n)** ignorando dettagli inessenziali come le costanti moltiplicative…
- Useremo a questo scopo la notazione asintotica O

Notazione asintotica O

- Diremo che f(n) = O(g(n)) se f(n) <= c g(n) per qualche costante c, ed n abbastanza grande



Esempi di Growth Rate di T(n)

Grossolaneamente….

Esempio1. al crescere di n, come cresce T(n)?

Istruzione di assegnamento $\mathbf{T}(n) = c$ *costante = O(1)*

Esempio 2: $\mathbf{T}(n) = c_1 n + c_2$ steps = O(n)

```
// Return position of largest value in "A"
static int largest(int[] A) {
  int currlarge = 0;  // Position of largest
  for (int i=1; i<A.length; i++)
    if (A[currlarge] < A[i])
       currlarge = i; // Remember pos
  return currlarge;   // Return largest pos
}
```

Esempi (cont.)

Esempio 3: $\mathbf{T}(n) = c_1 n^2 + c = O(n^2)$

```
sum = 0;
for (i=1; i<=n; i++)
  for (j=1; j<n; j++)
    sum++;
}
```

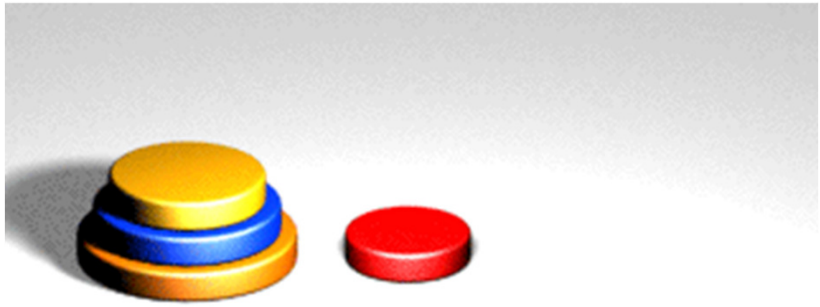
Esempi (cont.)

Esistono degli algoritmi che sono **intrinsecamente esponenziali**

- **Esempio**: un algoritmo che calcola le permutazioni di n elementi:

$$n! \sim n^n$$

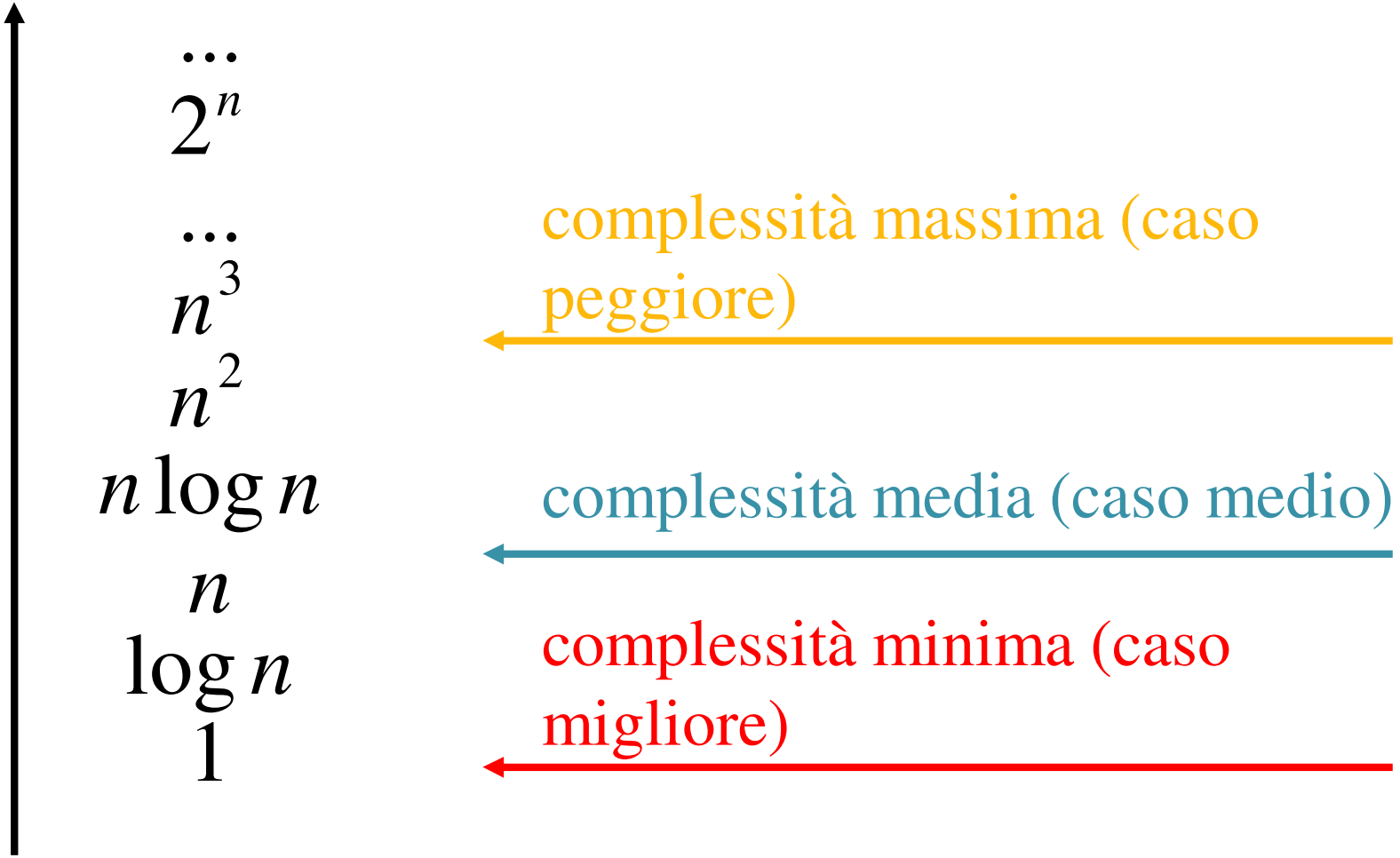
- **Esempio**: Torre di Hanoi: per spostare n dischi ci vogliono $2^n - 1$ mosse



Forma dell'input

Un algoritmo non ha, in generale, lo stesso comportamento per tutti gli input di una data taglia:

- *Esempio*: ordinare *n* oggetti può dipendere dal loro ordine iniziale (per esempio, potrebbero già essere ordinati!)



Progetto di algoritmi più efficienti

- Un'analisi più approfondita del problema permette di progettare algoritmi più efficienti
- **Esempio** (**Duplicati**): **verificaDup** ha tempo di esecuzione O(1) nel caso migliore e $O(n^2)$ nel caso peggiore, per una sequenza di n elementi.



```
algoritmo verificaDup(sequenza S)
    for each elemento x della sequenza S do
        for each elemento y che segue x nella sequenza S do
            if x = y then return true
    return false
```

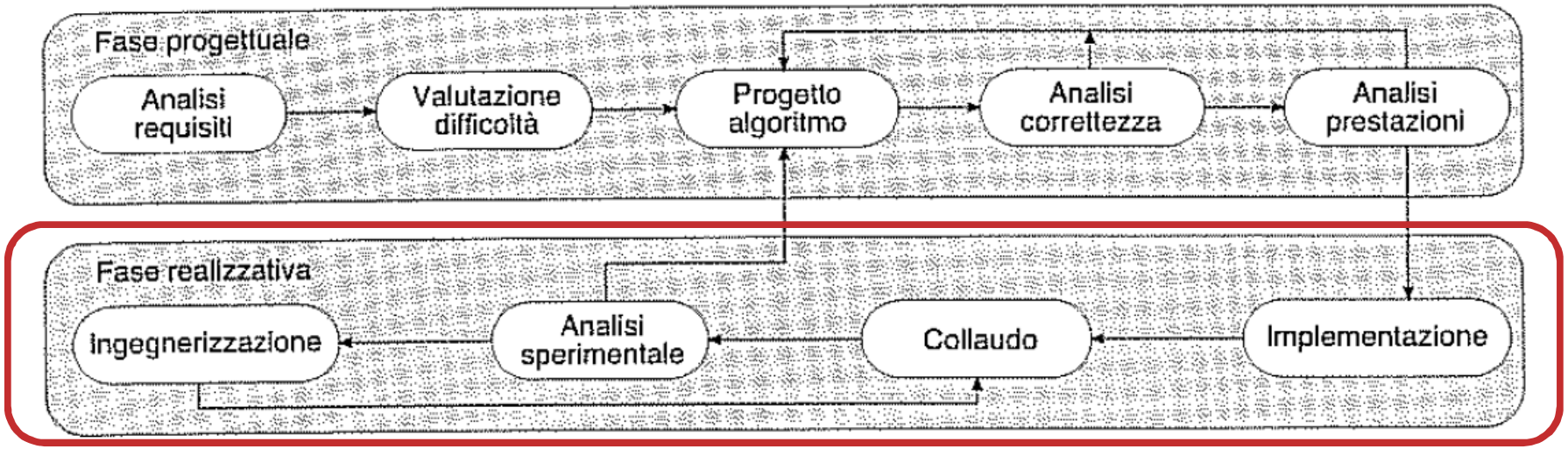
IDEA: Possiamo fare di meglio se la sequenza in input è ordinata (i duplicati sono consecutivi)!

Progetto di algoritmi più efficienti

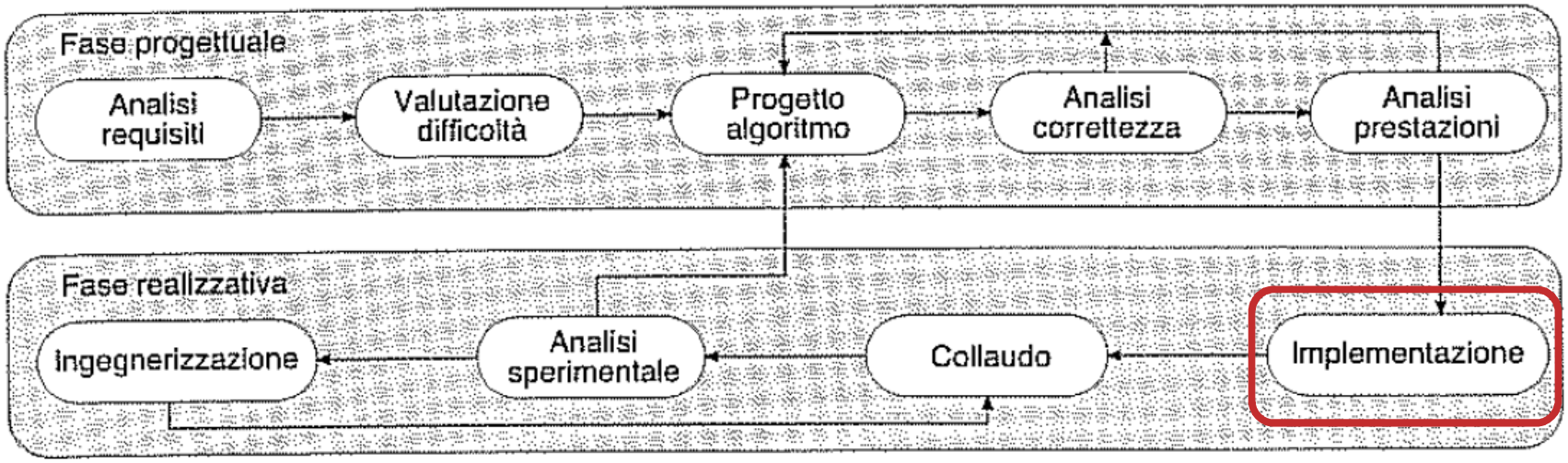
- **Esempio** (**Duplicati**): **verificaDupOrd** può essere implementato in modo da avere tempo di esecuzione O(n log n) nel caso peggiore:
  - Linea 1: Esistono algoritmi di ordinamento in grado di operare in tempo O(n log n) anche nel caso peggiore
  - Le linee 2-5 richiedono tempo O(n) nel caso peggiore

```
1 algoritmo verificaDupOrd(sequenza S)
2     ordina la sequenza S in modo non-decrescente
3     for each elemento x nella sequenza S riordinata, tranne l'ultimo do
4         sia y l'elemento che segue x in S
5         if x = y then return true
6     return false
```

- **Fase realizzativa**



- **Fase realizzativa**

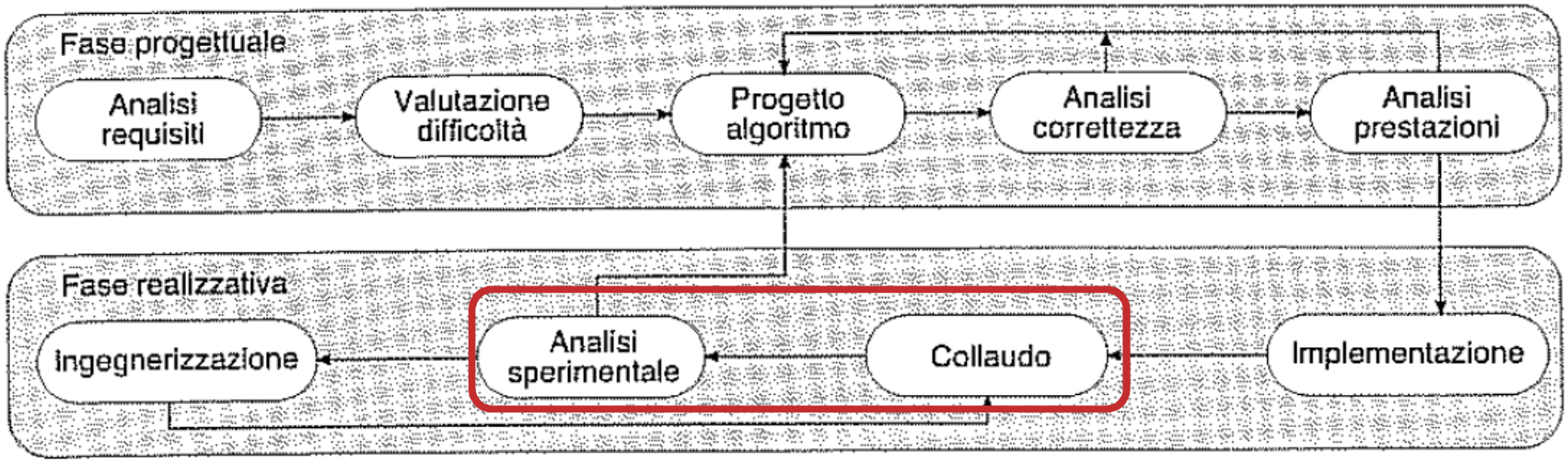


Implementazione in un linguaggio di programmazione

- Comporta molti dettagli implementativi non del tutto specificati nel pseudocodice
- *Esempio* (**Duplicati**): assumiamo che la sequenza S sia rappresentata come una lista (in Java un oggetto di una classe che implementa l'interfaccia **java.util.List** del **Java Collection Framework**)

```
public static boolean verificaDupList(List S) {
   for (int i = 0; i < S.size(); i++) {
      Object x = S.get(i);
      for (int j = i + 1; j < S.size(); j++) {
         Object y = S.get(j);
         if (x.equals(y)) return true;
      }
   }
   return false;
}
```

- **Fase realizzativa**

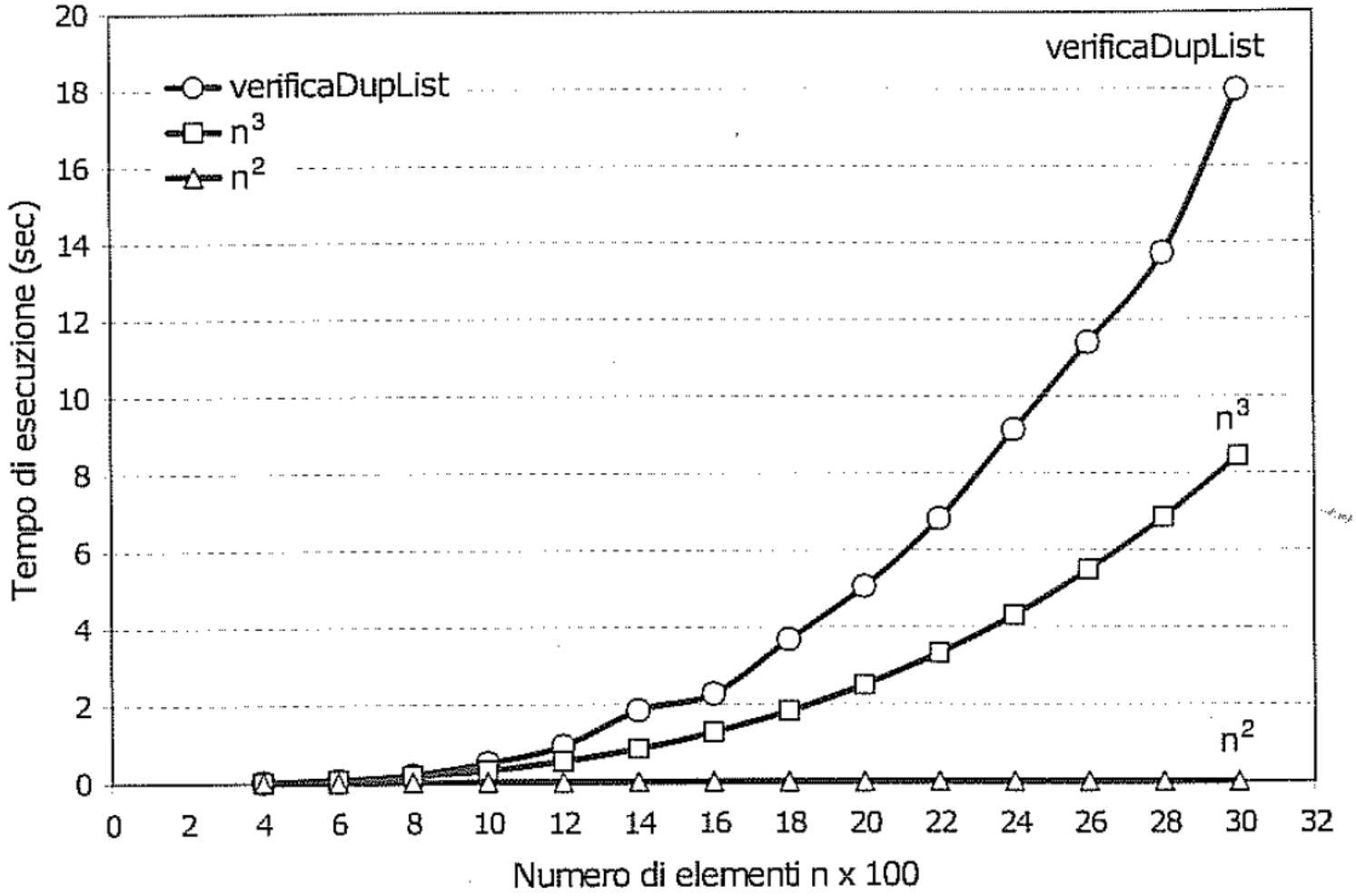


Collaudo e analisi sperimentale

- Una data implementazione va testata su data set reali per identificare eventuali errori implementativi
  - **Una data implementazione può risultare meno efficiente rispetto ai risultati dell'analisi teorica!**
  - In tal caso **occorre una messa a punto e re-ingegnerizzazione** opportuna che tenga conto delle caratteristiche del linguaggio e delle piattaforme di esecuzione
- Tale analisi sperimentale ci aiuta anche ad identificare le "**costanti nascoste**" (non considerate nell'analisi teorica) e a confrontare in modo più preciso algoritmi apparentemente simili

Collaudo e analisi sperimentale

- Esempio (Duplicati) La curva di **`verificaDupList`** non sembra somigliare alla funzione c n^2 ma alla funzione c n^3



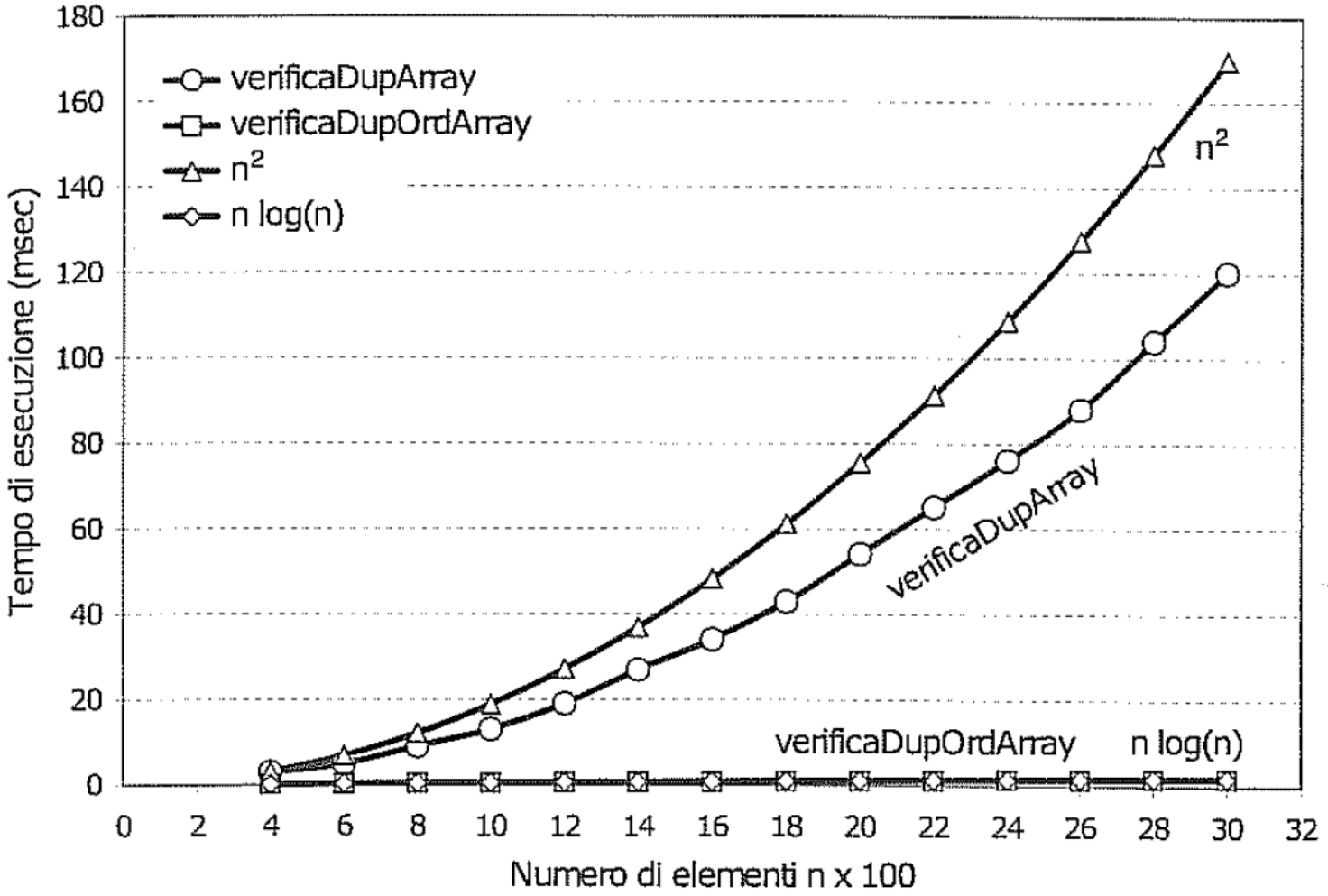
Collaudo e analisi sperimentale

- Esempio (Duplicati) L'inefficienza può essere eliminata implementando S come array.

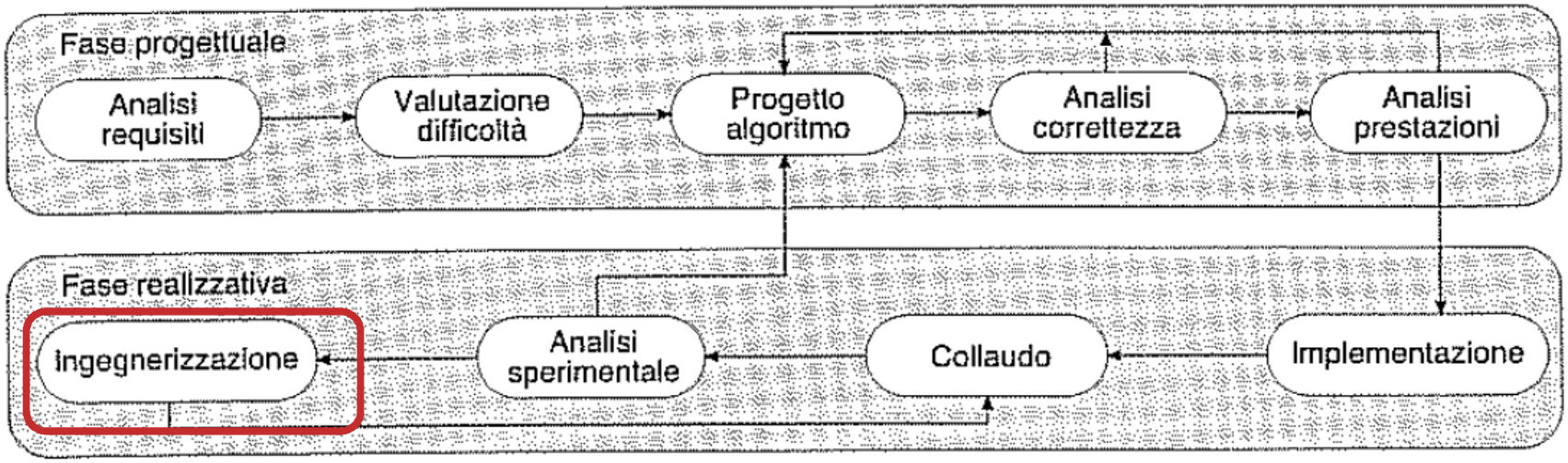
```
public static boolean verificaDupArray(List S) {
    Object[] T = S.toArray();
    for (int i = 0; i < T.length; i++) {
        Object x = T[i];
        for (int j = i + 1; j < T.length; j++) {
            Object y = T[j];
            if (x.equals(y)) return true;
        }
    }
    return false;
}
```

Collaudo e analisi sperimentale

- Esempio (Duplicati) Con **`verificaDupArray`** i risultati sono perfettamente allineati con la predizione teorica.



- **Fase realizzativa**



Ingegnerizzazione per ottimizzare/fare refactoring a livello di codice